Submodules Catalogue — `vasic-digital` + `HelixDevelopment`

Field	Value
Revision	1
Created	2026-05-20
Last modified	2026-05-20
Status	active
Status summary	Initial catalogue. Enumerates every owned-by-us submodule under the two canonical organisations per Universal §11.4.74 (submodule-catalogue-first discovery). 142 repos total (122 + 20). Categorised by capability so consuming projects can locate "is there already a Submodule for X?" in one glance. Re-generate via `gh repo list` + the script in §3 below.
Issues	none
Issues summary	—
Fixed	initial catalogue creation
Fixed summary	per user mandate 2026-05-20 follow-up to §11.4.76.
Continuation	Re-enumerate quarterly (or after any non-trivial repo creation/rename in either org).

Table of contents

- §1. Why this file exists
- §2. How to consume the catalogue

- §3. Regeneration
- §4. vasic-digital catalogue (122 repos)
- §5. HelixDevelopment catalogue (20 repos)

§ . Why this file exists

Per Universal §11.4.74 (Submodule-catalogue-first discovery + extend-don't-reimplement), every consuming project MUST survey these two organisations BEFORE scaffolding a new module. Without a single canonical inventory the survey becomes "open the GitHub page and skim 142 repos" — slow, easy to miss, error-prone.

This file is that inventory. It is the **first stop** for any "do we already have something that does X?" question. The full discipline is defined in `Constitution.md` §11.4.74; this file is the operational companion.

Per Universal §11.4.76, the `vasic-digital/containers` Submodule is the canonical container-orchestration substrate — every containerised workload across every consuming project goes through it.

§ . How to consume the catalogue

When scaffolding a new feature in any consuming project:

1. Find the closest capability match in §4 or §5 below.
2. If a match exists, add the submodule to the consuming project at the parent-root path (`<root>/<name>/` or `<root>/submodules/<name>/`) per Universal §11.4.28 (Submodules-As-Equal-Codebase).
3. Record the verdict in the tracker row: `Catalogue-Check: reuse <org/repo>@<sha>` or `Catalogue-Check: extend <org/repo>@<sha>` .
4. If extending: open a PR against the upstream Submodule (never duplicate in-project per §11.4.74).
5. If no match: record `Catalogue-Check: no-match <YYYY-MM-DD>` and justify in the PR description — operator-side validation re-checks the catalogue before approval.

The `no-match` path is rare — most "I need a small utility" needs are already covered by `digital.vasic.commons`-shaped modules below.

§ . Regeneration

Re-generate the lists in §4 / §5 by running:

```
gh repo list vasic-digital --limit 200 --json name,description,updatedAt > /tmp/v.json
gh repo list HelixDevelopment --limit 200 --json name,description,updatedAt > /tmp/h.json
```

then categorise via the python helper kept inline in this commit (`scripts/build_catalogue.py` is the planned canonical tool — currently inline-built per the §11.4.74 launch commit). Update mtime in the metadata table on every regen.

§ . vasic-digital catalogue (repos)

Container + lifecycle

- **`AgentWrapper`** — Wrap AI CLI Coding Agents in Docker containers
- **`Agentic`** — Graph-based agentic workflow orchestration
- **`AutoTemp`** — SCAFFOLD / WIP -- Benchmark-driven temperature auto-tuning orchestration
- **`BackgroundTasks`** — Background Tasks module
- **`HelixQA`** — AI-driven QA orchestration for multi-platform testing
- **`HyperTune`** — SCAFFOLD / WIP -- Hyperparameter tuning orchestration
- **`LLMOrchestrator`** — Headless CLI agent management for LLM orchestration
- **`tmux`** — Optimized + verified containerized tmux build — reproducible across hosts, jemalloc-aware, OOM-protected, 8-test verification gate. Reusable on any Linux system.

AI / LLM provider + agent

- **`Android-Toolkit`** — Commonly used set of abstractions and implementations.
- **`Benchmark`** — LLM benchmarking: SWE-bench, HumanEval, MMLU, leaderboard
- **`Chutes-Toolkit`** — Chutes Toolkit
- **`Embeddings`** — Generic reusable Go module: digital.vasic.embeddings
- **`I-LLM`** — SCAFFOLD / WIP -- Introspection layer for LLM providers
- **`JVM-Toolkit`** — Commonly used set of abstractions and implementations.
- **`LLMGateway`** — (no description)
- **`LLMOps`** — LLM operations: evaluation, experiments, datasets, prompt versioning
- **`LLMProvider`** — Shared LLM provider interface, 40+ provider adapters, retry, circuit breaker, health monitoring
- **`LLMsVerifier`** — Benchmark and verify LLMs
- **`Memory`** — Generic reusable Go module: digital.vasic.memory
- **`Normalize`** — Adversarial-input canonicalisation library for defensive LLM guardrail pipelines
- **`Open-Rag-Material-Android`** — Material for RAG for Android development
- **`Optimization`** — Generic reusable Go module: digital.vasic.optimization
- **`Planning`** — AI planning algorithms: HiPlan, MCTS, Tree of Thoughts
- **`RAG`** — Generic reusable Go module: digital.vasic.rag
- **`RedTeam`** — YAML-driven adversarial prompt fixture harness for defensive LLM guardrail regression testing
- **`SelfImprove`** — AI self-improvement: reward modelling, RLHF, optimizer
- **`SiliconFlow-Toolkit`** — SiliconFlow Toolkit
- **`SkillRegistry`** — CLI agent skill registration and management for AI agent systems
- **`Storage`** — Generic reusable Go module: digital.vasic.storage
- **`TOON`** — Generic reusable Go module: digital.vasic.toon - Token-Oriented Object Notation wrapper

- `ToolSchema` — Generic tool schema definition, validation, and execution for AI agent tool systems
- `VectorDB` — Generic reusable Go module: digital.vasic.vectordb
- `VisionEngine` — Computer vision and LLM Vision for UI analysis and navigation
- `conversation` — Conversation context management, infinite context compression, and event sourcing for AI agents

Messaging + observability + storage

- `Document` — Go document model with format detection and change tracking
- `EventBus` — Generic reusable Go module: digital.vasic.eventbus
- `Filesystem` — digital.vasic.filesystem - Reusable Go module
- `Formatters` — Generic reusable Go module: digital.vasic.formatters
- `I18n` — digital.vasic.i18n - Generic internationalization Go module
- `Lazy` — Generic reusable Go module: digital.vasic.lazy - Lazy loading with sync.Once generics
- `MCP_Module` — Generic reusable Go module: digital.vasic.mcp
- `Messaging` — Generic reusable Go module: digital.vasic.messaging
- `Plugins` — Generic reusable Go module: digital.vasic.plugins
- `Streaming` — Generic reusable Go module: digital.vasic.streaming
- `cache` — Generic reusable Go module: digital.vasic.cache
- `concurrency` — Generic reusable Go module: digital.vasic.concurrency
- `config` — digital.vasic.config - Reusable Go module
- `database` — Generic reusable Go module: digital.vasic.database
- `discovery` — digital.vasic.discovery - Reusable Go module
- `http3` — Generic Go module wrapping quic-go/http3 for net/http.Handler servers — drop-in HTTP/3 support
- `mdns` — Generic Go module for RFC 6762/6763 mDNS service announcement and discovery — drop-in LAN service registration
- `observability` — Generic reusable Go module: digital.vasic.observability
- `ratelimiter` — digital.vasic.ratelimiter - Reusable Go module

- `recovery` — Generic reusable Go module: digital.vasic.recovery - Application-level fault tolerance
- `tracker_sdk` — Generic, tracker-agnostic SDK primitives: mirror manager, plugin registry, testing harness. Used by the Lava project (https://github.com/milos85vasic/Lava) to support multiple torrent trackers.

Auth + security + middleware

- `Claritas` — SCAFFOLD / WIP -- System-prompt extraction detection
- `GandalfSolutions` — SCAFFOLD / WIP -- Read-only solutions archive for prompt-leak-defense testing
- `LeakHub` — SCAFFOLD / WIP -- Prompt-leak corpus / defensive-use fixtures (red-team training)
- `Ouroborous` — SCAFFOLD / WIP -- Recursive/self-referential safety patterns
- `Veritas` — SCAFFOLD / WIP -- Truth/verification auxiliary
- `auth` — Generic reusable Go module: digital.vasic.auth
- `middleware` — digital.vasic.middleware - Reusable Go module
- `security` — Generic reusable Go module: digital.vasic.security

Cross-platform (KMP) modules

- `Auth-KMP` — Kotlin Multiplatform OAuth2 authentication: flows, token management, secure storage interface
- `Concurrency-KMP` — Kotlin Multiplatform concurrency utilities: lazy loading, platform synchronization, flow-based loaders
- `Config-KMP` — Kotlin Multiplatform storage configuration types for network protocols
- `Database-KMP` — digital.vasic.database - KMP network storage database interfaces and entity types
- `Document-KMP` — Kotlin Multiplatform document model with format detection and change tracking
- `Formatters-KMP` — Cross-platform KMP text format detection, parsing interfaces, and format registry

- `RateLimiter-KMP` — Kotlin Multiplatform rate limiting: semaphore, token bucket, adaptive, throttler
- `Security-KMP` — Kotlin Multiplatform secure storage: AES encryption, platform Keychain/KeyStore integration
- `Storage-KMP` — digital.vasic.storage - KMP network storage service interfaces and abstractions
- `UI-Components-KMP` — Kotlin Multiplatform UI components: theme system, animations, accessibility utilities for Compose

TypeScript / React + Web

- `API-Client-TS` — Generic REST API client for TypeScript
- `Auth-Context-React` — React AuthProvider and useAuth hook for Catalogizer authentication with React Query
- `Calling-Engine-TS` — Generic SIP.js + WebRTC calling engine
- `Catalogizer-API-Client-TS` — Type-safe TypeScript client for the Catalogizer API with axios, auth, retry, and per-domain services
- `Collection-Manager-React` — React collection management components for Catalogizer
- `Dashboard-Analytics-React` — React dashboard and analytics components for Catalogizer
- `I18n-Client-TS` — Generic client-side i18n wrapper around i18next
- `Media-Browser-React` — React entity browser components for Catalogizer media browsing
- `Media-Player-React` — React media player component for Catalogizer entity playback
- `Media-Types-TS` — Shared TypeScript type definitions for Catalogizer media entities, auth, and API
- `State-Management-TS` — Generic Redux Toolkit state management utilities
- `Testing-Utils-TS` — Generic React + Redux testing utilities
- `UI-Components-React` — Reusable TypeScript/React module
- `WebSocket-Client-TS` — Reusable TypeScript/React module

Testing + QA + benchmarking

- **`Panoptic`** — A comprehensive tool for automated testing, UI recording, and screenshot capture across web, desktop, and mobile applications.
- **`ReplayBuffer`** — Reusable ReplayBuffer module for visual testing and automation
- **`ScreenDiff`** — Reusable ScreenDiff module for visual testing and automation
- **`TrainingCollector`** — Reusable TrainingCollector module for visual testing and automation
- **`VisualRegression`** — Reusable VisualRegression module for visual testing and automation
- **`challenges`** — (no description)

Apps / tools / experimental

- **`ATMOSphere-Android-15`** — (no description)
- **`ATMOSphere-Android-15-Main`** — Android 15 AOSP for Orange Pi 5 Max
- **`Asinka`** — Interprocess Objects Syncrhronization Library: Asinka (Асинка)
- **`Assets`** — Generic, reusable Go module for lazy asset loading with strategy-based resolution
- **`Catalogizer`** — Advanced Multi-Protocol Media Collection Management System
- **`Courses-Creator`** — Courses creator toolkit
- **`DocProcessor`** — Documentation processing and feature map extraction for QA automation
- **`Entities`** — Generic media entity system: title parsing, media types, hierarchy detection, and aggregation primitives for Go
- **`Games`** — Vasic Digital Games
- **`GrabTube`** — Tube services downloader
- **`HelixAgent`** — (no description)
- **`Integrations`** — Integrations
- **`Media`** — digital.vasic.media - Reusable Go module
- **`Models`** — (no description)

- **`Network-Binder`** — Bind multiple internet connection endpoints into single one using mptcp.
- **`OOM-Protect`** — Workstation OOM hardening for systemd Linux
- **`PliniusCommon`** — SCAFFOLD / WIP -- Common types/errors library (shared foundation for the 8 sibling modules)
- **`Proxy`** — Proxy server
- **`Pure_v2_Firmware`** — Firmware for BeagleBone
- **`SDK`** — SDK
- **`ShareConnect`** — Share the downloadable URLs to remote (or local) processing endpoints
- **`TransmissionConnect`** — Remote control for Transmission BitTorrent client and integration with ShareConnect
- **`Watcher`** — digital.vasic.watcher - Reusable Go module
- **`Yole`** — Text editor, Notes & ToDo - Markdown, todo.txt, plaintext, math, and much more ...
- **`ffmpeg-kit`** — FFmpeg Kit for applications. Supports Android, Flutter, iOS, Linux, macOS, React Native and tvOS. Supersedes MobileFFmpeg, flutter_ffmpeg and react-native-ffmpeg.
- **`qBitConnect`** — Control qBittorrent from any device and integrate with ShareConnect

Misc / scaffolds

- **`Herald`** — Ingesting system events and reliably fanning them out to multiple notification channels so every alert reaches the right destination without confusion
- **`containers`** — (no description)
- **`vasic-digital.github.io`** — (no description)

§ . HelixDevelopment catalogue (repos)

- **`DebateOrchestrator`** — Multi-agent debate orchestration library — Phase 1 reconstruction (7 real + 5 stub sub-packages; see RECONSTRUCTION_ROADMAP.md)
- **`DocProcessor`** — Documentation processing and feature map extraction for QA automation
- **`Helix-Game-1`** — First Stake game from HelixDevelopment
- **`Helix-Game-Demo`** — (no description)
- **`HelixAgent`** — LLMs Agent
- **`HelixBuilder`** — AI powered application building pipeline
- **`HelixCode`** — AI Coding Agent
- **`HelixConstitution`** — Constitution, AGENTS.MD and CLAUDE.MD universal agent rules and constraints
- **`HelixGitpx`** — Helix Git Proxy eXtended
- **`HelixLLM`** — Helix LLM - Local running super model
- **`HelixMemory`** — Super Memory Provider — Unified Cognitive Memory Engine fusing Mem0 + Cognee + Letta for HelixAgent
- **`HelixPlay`** — Ultimate gaming experience!
- **`HelixSpecifier`** — Spec-Driven Development Fusion Engine - Unified specification intelligence fusing SpecKit + Superpowers + GSD for HelixAgent AI debate ensemble
- **`HelixTranslate`** — A high-performance, enterprise-grade universal ebook translation toolkit supporting any ebook format and any language pair, featuring multiple translation engines, REST API with HTTP/3 support, and real-time WebSocket events.
- **`LLMOrchestrator`** — Headless CLI agent management for LLM orchestration
- **`LLMProvider`** — Shared LLM provider interface, 40+ provider adapters
- **`Projects`** — Helix projects
- **`Stake-Tetris`** — Stake Tetris
- **`VisionEngine`** — Computer vision and LLM Vision for UI analysis and navigation

- `helixqa` — AI-driven QA orchestration for multi-platform testing